

Automated Theorem Proving: Language Model Based
Approaches to Tackle Undergraduate & High School
Competitions in Mathematics

Leo Gordon

Haverford College

December 19th 2025

Under the Supervision of Prof. Steven Lindell

Contents

1	Abstract	4
2	Introduction	4
3	Motivation	6
4	Background	7
4.1	Upcoming sections	9
5	Taxonomy of LLM based solutions	9
5.1	Off-The-Shelf LLMs	9
5.2	Fine tuned LLMs	10
5.3	Geometric Searchers	13
5.3.1	AlphaGeometry2	14
5.3.2	Seed-Geometry	15
5.4	LLM-Systems	15
5.4.1	Tencent Decoupled reasoning	16
5.4.2	Seed-Prover	20
5.4.3	Hilbert	21
5.4.4	Aristotle	25
5.4.5	Ax-Prover	26
5.4.6	AlphaProof	28
6	Problems with Benchmarks in ATP	29
6.1	Benchmarks Commonly Used for ATP	30
6.2	Data Contamination for all systems	31
6.3	Data Contamination in ATP	31
6.4	Reproducibility Concerns more broadly	32

7	Future directions	33
7.1	Documenting Scalability in System-Agnostic Techniques	33
7.2	Reproducing Papers and Comparing Results	34
7.3	Implementing papers that propose better benchmarking	34
	Bibliography	36

1 Abstract

Recent automatic proving systems using Large Language Models (LLMs) like AlphaProof and Hilbert report superhuman performance on math olympiad benchmarks (AIME, IMO, Putnam), and silver medal performance on IMO competitions. This thesis critically reviews these claims, questioning whether this is genuine reasoning or sophisticated pattern matching due to data contamination in test sets like MiniF2F and PutnamBench. This thesis additionally proposes a taxonomy of current approaches—from off-the-shelf models to complex systems and evaluates the compelling evidence for benchmark contamination and the field’s reproducibility challenges. This review urges a more cautious interpretation of progress and outlines future directions for more robust evaluation, while also highlighting the clear success and promise that these systems contain.

2 Introduction

In 2016, Google released the world-changing paper *Attention is All you Need* [20]. Originally intended for use in translation tasks, the transformer took the world by storm, eventually resulting in the widespread adoption and implementation of Large Language Models we see today. Large Language Models (or *LLMs* for short) have been adopted into virtually every facet of human life, as scientists and researchers scramble to determine how it can be applied to their fields. One such field is the application of Language Models in mathematics competitions or olympiads such as: the AIME, IMO, and Putnam exam.

Math competitions represent a unique challenge for artificial intelligence. Unlike rote computational tasks or even standard university-level math, olympiad level competitions require a high level of creativity, abstract reasoning, and the ability to synthesize many disparate mathematical concepts into a novel proof strategy, without access to any outside resources. Additionally, the problems are often designed to be non-standard, thus requiring insights that cannot be derived from following simple or widespread algorithms. For the above reasons, AI performance on these tests has become the de facto benchmark for evaluating these systems.

In recent years, a rapid succession of papers have claimed significant and superhuman performance on this topic, known as Automated Theorem Proving (ATP). Systems such as DeepMind’s *AlphaGeometry* [6] and Apple’s Hilbert [19] have reported results on established benchmarks like the IMO and PutnamBench, that would, in some cases, surpass top human competitors, winning gold medals in some years. These outstanding results have led to excitement and trepidation, fueling speculation about what the future of the field of mathematics as a whole will look like, and the obsolescence of human intellect.

However, these highly-publicized results require much scrutiny. Beyond the surface of the reported near-perfect performances, many critical questions remain regarding the true generalization capabilities of these models. A primary concern, highlighted by Khatibi et al. [12] is the potential for ”data contamination”, as highlighted by Balunović et al. [2] where problems and their solutions from popular benchmarks may have been included in the vast training corpora of these models. This raises a fundamental question: are these systems genuinely ”reasoning,” or are they performing a sophisticated form of pattern matching on familiar problems? There is strong reason for concern, as evidenced by Balunović et al. [2], where models were tested on questions that came from after models’ training cutoffs, and the results plummeted in precision.

This thesis aims to address these issues by providing a comprehensive review and analysis of the current landscape of LLM-Based mathematical prover systems and problem solvers. The primary goals are the following:

1. To propose a clear taxonomy of existing approaches, differentiating between larger systems and fine-tuned models, as well as domain specific methods like geometric solvers.
2. To critically evaluate the benchmarks used for measuring system performance, considering the compelling evidence of data contamination and its implications for reported results,
3. To chart the chronological progression of the key ideas that have led to the current state of the art.
4. To propose possible techniques that will advance these systems further.

Ultimately, this thesis will serve as a foundational review, urging a more cautious and nuanced interpretation of the progress in automated mathematical reasoning, and outlining more robust avenues for future evaluation.

3 Motivation

The field of LLM-Based proving systems is currently at a critical juncture. Systems like DeepMind’s *AlphaGeometry* [6] and Apple’s Hilbert [19] have published results claiming superhuman performance on prestigious olympiad benchmarks scoring results that would easily give gold medals on the IMO, and dominate the Putnam exam.

As a somewhat obvious result of these purported claims, there has been a considerable amount of attention, hype and general excitement around these systems, especially with respect to how these models are eclipsing humans. As stated previously, these are really hard problems to solve, and they hint at these systems being able to do far more than what we currently anticipate. This narrative, however compelling, demands intense scrutiny.

The primary motivation for this thesis is to address the disconnect between these reported outstanding results and the growing body of evidence suggesting that these results are exaggerated, and many times unreliable. Many papers, such as, Khatibi et al. [12] and Balunović et al. [2] provide compelling evidence that ”data contamination” is rampant in these systems, or that many of the questions on the benchmarks were likely ingested during the general training of Off-The-Shelf LLMs. So are we even celebrating genuine progress in the field, or are we looking at statistical pattern matching from a contaminated test set?

Furthermore, this field has progressed rapidly over the past 18 months to produce paper after paper, introducing an entirely new type of system, or claiming better results. No one has yet looked back on the entire body of study, to critically determine the differences between all of the existing approaches. This analysis will reveal what can be combined, what can be improved upon, and any logical next steps to improve performance.

4 Background

The field of technologies related to language models is rapidly evolving, with quickly changing terminology and definitions. Here are some select terms that will be useful for this paper:

LLM

Refers to a Large Language Model, characterized by having a large size (in the billions or trillions of parameters) and having a deep learning/transformer architecture. These models are generally trained on massive corpora of texts, encompassing nontrivial fractions of all publicly available information on the internet.

Fine-Tuning

The process of adapting a general-purpose, pre-trained model (like an LLM) for a specific task or domain. This is done by taking specific examples from the subdomain that the model is being fine-tuned to, and continuing training for this subset.

LLM System

Much like a distributed system encapsulates many different programs being run on different computation instances, an LLM System in the context of this thesis refers to an interdependent and connected system of LLM calls where each call has a specific role.

Data Contamination

A critical methodological error in machine learning where information from the evaluation or test dataset inadvertently "leaks" into the training dataset. This leads to an overestimation of the model's performance and invalidates the evaluation results. Sort of like looking at the answers to an exam before taking it.

Automated Theorem Proving (ATP)

A subfield of artificial intelligence and mathematical logic focused on the development of computer programs to automatically discover, construct, and verify formal proofs for mathematical theorems.

SOTA:

Literally means *State of The Art* or, the best current approach.

Pass@N:

The number of trials run for a given task before yielding a success or failure. If at least one of the trials is successful then the task is successful at N trials.

LLM-As-A-Judge

A technique involving the use of a Large Language Model to automatically evaluate the correctness of mathematical proofs or reasoning steps. Standard, off-the-shelf LLMs have been shown to be unreliable for this task, often failing to identify logical flaws when assessing a complete proof [9]. Research, such as that by Guo et al. [9], has proposed model-agnostic verification systems to improve accuracy. This is achieved by providing the "judge" model with very small, focused context windows, compelling it to rigorously evaluate only one specific part of a proof at a time rather than being misled by the proof's broader structure.

Lean 4 Overview

Lean 4 is a functional programming language and interactive theorem prover used to write formal, machine-verified mathematical proofs. While understanding the exact syntax is not necessary for this literature review, there are a few critical features of its syntax required to understand the systems henceforth.

1. A **have** statement is a keyword used inside a proof to introduce an intermediate step or lemma. It allows you to state a new claim, provide its proof in a nested block, and then use that proven claim to help prove the main goal.
2. A **sorry** statement is a placeholder tactic that tells the Lean compiler to accept a proof goal as complete without an actual proof. It is used to temporarily bypass difficult steps and continue working on other parts of a larger proof, but it ultimately signifies an incomplete or unverified part of the theorem. These are generally used in the earliest stages of generating a proof, and must be completely resolved by the end of the proof generation process.

A critical enabler for this line of research has been the development of environments and toolkits like **LeanDojo** [21]. LeanDojo provides a comprehensive toolkit for interacting with the Lean

proof assistant, enabling researchers to extract proof states, premises, and tactics from existing Lean projects like Mathlib. More importantly, it established a framework for retrieval-augmented language models, where a model’s generation process is guided by ”retrieving” relevant theorems and definitions from the vast Mathlib library. This retrieval-augmented generation (RAG) approach, facilitated by tools like LeanDojo, became a foundational component for many of the sophisticated provers and systems that followed.

4.1 Upcoming sections

In [section 5](#), we give a general taxonomy of LLM based solutions, charting who invented what, and the similarities and differences of individual approaches. In [section 6](#) we explore the role of benchmarks in these proving systems, and their shortcomings. Lastly in [section 7](#), we discuss logical future directions for the field of Automated Theorem Proving.

5 Taxonomy of LLM based solutions

5.1 Off-The-Shelf LLMs

What does it even mean for a Language Model to solve a math problem? The trivial answer to this question is to just ask a model to generate a proof to a problem in natural language, based on a problem statement. This is the core idea that separates LLM-Based solutions from non LLM-Based solutions. We are trying to take advantage of the insights gained from training these models to yield a genuine creative power for solving novel and hard problems.

With the raw reasoning power that these models have accumulated over the past few years, it is therefore no surprise that their strength has grown exponentially on mathematical reasoning problems, with one study reporting that Gemini 2.5 Pro had a 49% pass rate [\[23\]](#) on the Mini-F2F benchmark Zheng et al. [\[22\]](#), which comprises a mix of undergraduate and high-school olympiad level problems. How do these base models even do this?

The answer is fairly simple. These models aren't actually synthesizing anything new, they achieve this performance primarily through their vast training data, which often includes extensive mathematical corpora, online problem-solving forums, and digitized textbooks. Their process is not one of symbolic reasoning in the human sense, but of sophisticated pattern recognition. In fact, there is even evidence that the patterns for solving these problems all exist because the AI may have been trained on these problems, a problem called *Data Contamination*, which is discussed in detail in the next section. However, these models show relatively decent performance when tested on truly new problems after their training cutoffs, such as Gemini 2.5's score of 32% on this year's IMO problems, and Balunović et al. [2], with the confidence interval including the purported 50% figure.

However, we are left with plenty of room for improvement with these numbers. An analysis of the questions with which LLMs struggle yields clear pain points with which these basic models struggle. Petrov et al. [14] observed that unlike humans, which generally fail because they cannot write an answer of any kind, these models fail because they pass off flawed logic, produce solutions frequently using unjustified reasoning steps, have incorrect rationales, or misinterpret previous progress. They also tend to claim critical steps as trivial, even when their validity was crucial. Lastly, [14] observed that these models generally lack a sense of creativity in their solutions, with models usually following similar chains of reasoning with very little deviation. All of these factors combined lead to poor performance, but also created room for improvement in these systems discussed later on.

5.2 Fine tuned LLMs

While Off-The-Shelf LLMs provide a powerful baseline, their general-purpose training is not optimized for the exact precision required by formal proof languages. The next major leap in performance came from creating specialized models, or "fine-tuned LLMs," which are trained specifically on massive, synthetically-generated datasets of formal proof problems. This approach, often called "expert iteration," involves using models to generate new proofs, verifying them, and adding the successful ones back into the training data to create a progressively stronger "expert"

model.

A foundational paper in this area is **DeepSeek-Prover-V2** [15]. This work established a "cold-start" data synthesis pipeline that effectively bridges the gap between informal, human-like reasoning and formal, machine-verifiable code. The process begins by using a powerful generalist model (DeepSeek-V3) to read a complex problem and generate a high-level, natural language proof sketch, simultaneously decomposing the problem into a series of simpler subgoals. A smaller, more specialized 7B parameter prover model is then tasked with solving these more manageable subgoals. The successful formal proofs for these subgoals are then synthesized and combined with the original high-level reasoning sketch, creating a rich new data point. This dataset, which unifies informal strategy with formal execution, is then used to train the final, large-scale specialist model (DeepSeek-Prover-V2-671B) using Reinforcement Learning (RL). This paper set a new SOTA at its time, demonstrating the power of this "sketch-and-solve" data pipeline. DeepSeek-Prover-V2-671B was able to achieve an 82% pass rate at pass@32 using chain of thought, which improved to 89% with 8192 trials. In non chain of thought mode, it only achieved 74% on pass@32. DeepSeek-Prover-V2-7B achieved 75.6% pass rate on pass@32 chain of thought, while not using chain of thought, it achieved a 68% pass@32 rate. For such a small model, these results are very impressive.

The next major advancement, **Goedel-Prover-V2** [13], built directly upon this established expert iteration and RL pipeline. In fact, it even used the DeepSeek-Prover models to help generate its initial dataset. The innovations of Goedel-Prover-V2 demonstrated that a smarter training process could dramatically outperform raw model scale; its 8B parameter model successfully outperformed the 671B DeepSeek-Prover-V2 on the MiniF2F benchmark. This was achieved through three key innovations:

1. **Verifier-Guided Self-Correction:** This was the most significant leap. Instead of just generating a proof in a single pass, the model iteratively revises its own attempts. It generates a proof, receives direct error feedback from the Lean compiler (the "verifier"), and then uses that feedback to generate a corrected proof. This human-like feedback loop proved far more effective than simple generation.

2. **Scaffolded Data Synthesis:** A more advanced form of data generation. If a proof for a problem failed, the model was prompted to generate simpler related problems. If a proof succeeded, it was prompted to generate harder variants. This created a "scaffolded" curriculum of increasing difficulty.
3. **Model Averaging:** A training technique to mitigate the loss of output diversity that often occurs during late-stage RL training. By averaging the weights of different model checkpoints, the final model retained a wider variety of problem-solving strategies.

Goedel-Prover-V2's success showed that the path forward was not just bigger models, but models that could learn from their own mistakes. It achieved 90.4% accuracy at pass@32 with self-correction on.

Concurrently, another line of research focused on a different, critical weakness: the *fragility* of these models. Tian et al. [17] introduced **EvolProver**, a 7B non-reasoning model designed to improve generalizability and robustness. This work, which used DeepSeek-Prover-V1.5 as its base model, argued that existing models were too easily broken by minor, semantically-irrelevant transformations of a problem statement. The entire focus of this paper was on data augmentation, introducing three novel techniques:

- **EvolAST:** A data-augmentation method to create new training data. It parses a formal statement (like $x+y=z$) into an Abstract Syntax Tree (AST), applies logical equivalence rules (like commutativity, $y+x=z$), and then recompiles the new AST into a syntactically different but semantically identical problem.
- **EvolDomain:** An LLM-driven method to "evolve" problems by translating their core logic across different mathematical domains (e.g., translating a number theory problem into a geometry problem).
- **EvolDifficulty:** An LLM-driven method, similar to Goedel's scaffolding, to generate new theorems with a wider range of difficulty.

EvolProver demonstrated that by training on this highly diverse and robust dataset, a small, efficient 7B *non-reasoning* model could achieve SOTA performance for its class, even outperforming several larger *reasoning* models on benchmarks like FormalMATH-Lite. This progression—from DeepSeek’s “sketch-and-solve” data, to Goedel’s “self-correcting” loop, to EvolProver’s “robustness-focused” data—charts a clear path from brute-force synthesis toward more intelligent and resilient training paradigms. In the end, EvolProver achieved a 69.8% rate at pass@32, which is the SOTA for no chain-of-thought.

It is important to note that a lack of standardized evaluation protocols complicates direct comparisons between models like Goedel-Prover and EvolProver. The papers report performance using different sample budgets (e.g., pass@32, pass@1024, pass@8192) and methodologies, such as with or without Chain-of-Thought or verifier-guided self-correction. Furthermore, the internal development process—such as the number of training iterations conducted before selecting the final published model—remains opaque. This opacity raises concerns about selective reporting, analogous to “p-hacking,” where only the most favorable results might be published. However, even accounting for this methodological skepticism, the significant performance gains across these models clearly indicate substantial progress in the field, affirming the high capability of these fine-tuned provers.

5.3 Geometric Searchers

While the fine-tuned models discussed previously attempt to solve problems by generating a complete formal proof in one pass, a different and highly successful category of systems tackles the specific domain of geometry. These **geometric searchers** are complex, multi-component architectures known as **neuro-symbolic systems**. This approach is defined by a synergistic pairing of a neural component (an LLM) and a symbolic component (a logical reasoning engine). As described by Chervonyi et al. [6], the LLM provides a “creative” function, suggesting novel auxiliary constructions needed to solve a problem, while the symbolic engine provides a “reliable” deductive function, rigorously deriving all possible facts from a given set of geometric premises [6]. Two of the

most prominent examples of this architecture are Google’s AlphaGeometry2 and ByteDance’s Seed-Geometry.

5.3.1 AlphaGeometry2

AlphaGeometry2 (AG2) is a system that surpassed the performance of an average IMO gold medalist in geometry [6]. Its architecture is a significant evolution from its predecessor, AlphaGeometry (AG1), and is built on two key innovations: an improved symbolic engine and a novel search algorithm. The AG2 LLM was trained on a massive, synthetically generated dataset of 100 million examples allowing it to develop a powerful intuition for suggesting useful auxiliary constructions.

The core of AG2 is its symbolic engine, **DDAR** (Deductive Database Arithmetic Reasoning), which is responsible for computing the “deduction closure”—the set of all provable facts given the problem’s starting conditions. The original AG1 used DDAR1, which was slow; for instance, its search for similar triangles had a time complexity of $O(N^8)$. DDAR2, the new engine in AG2, is a “stronger and faster symbolic engine” due to several upgrades. Algorithmically, it uses more efficient methods like hashing to find similar triangles and cyclic quadrilaterals. Physically, the engine’s core was rewritten in C++ and exposed to Python via pybind11, resulting in a speedup of over 300x in benchmark tests (from ≈ 1180 seconds to ≈ 3.4 seconds). Critically, DDAR2 also added the ability to handle “double points,” a technique that allows the system to prove that two points with different definitions (e.g., X as the intersection of lines a, b and X' as the intersection of line a and circle w) are, in fact, the same point. This enables powerful proof by reformulation. AG1 used a simple beam search to find proofs. AG2 introduces a far more robust algorithm called **Shared Knowledge Ensemble of Search Trees (SKEST)**. In this system, multiple, differently-configured search trees (e.g., one that adds one auxiliary point at a time, one that adds multiple) are executed in parallel, often using different LLMs. [6]

5.3.2 Seed-Geometry

Similar to AlphaGeometry, Seed-Geometry [5] is a **neuro-symbolic system** designed to solve complex geometry problems. It was developed as a component of the larger **Seed-Prover** system, which is a "lemma-style whole-proof reasoning model" designed to work with the formal language Lean. The authors of Seed-Prover noted a "lack of sufficient geometry support in Lean" and therefore built Seed-Geometry as a "dedicated geometry reasoning engine" to handle this domain. Like AG2, Seed-Geometry is a neuro-symbolic system that follows a forward-chaining design . It pairs a high-performing LLM from the "Seed family" with a "specialized reasoning engine". As a proprietary system, we don't know many of the exact details. A core theme shared with AG2 is the emphasis on speed and data. The Seed-Geometry engine was also rewritten in C++ (from a Python implementation in its predecessor, TongGeometry) for a 100-fold speed increase. This speed was necessary to power its "extensive search" data generation pipeline, which created a massive repository of over **230 million unique geometry problems**. This vast synthetic dataset was then used to train the Seed LLM policy model, granting it the expert intuition needed to guide the symbolic prover.

In summary, both AG2 and Seed-Geometry demonstrate that SOTA performance in olympiad geometry is currently achieved not by a single monolithic LLM, but by a complex system. These systems pair an LLM trained on massive synthetic datasets (100M-200M+ problems) for creative intuition, with a highly-optimized, custom-built symbolic engine (often in C++) for fast and rigorous logical deduction.

5.4 LLM-Systems

In the previous subsections of this chapter, we have seen many individual single LLM solutions that can be employed to tackle proving math problems. However, there are fundamental

Model	Accuracy	Compute
Hilbert	72% [462]	avg. pass@1840
Seed-Prover	51% [329]	medium
Ax-Prover [†]	14% [92]	pass@1 [‡]
15Goedel-Prover-V2 [†]	13% [86]	pass@184
DeepSeek-Prover-V2 [†]	7% [47]	pass@1024
DSB [†]	4% [323]	pass@128

limits with these single model approaches. Models work best on solving specific subproblems, and fulfilling specific and well defined roles. These systems will serve to further specialize the roles of these models in creating proofs. In this section, therefore, we will increase the complexity of our approaches to incorporate different instances of LLMs communicating with one another and acting as agents fulfilling specific roles. Furthermore, in this section, our focus shifts entirely away from proofs in natural language, and instead entirely towards formalisms in **Lean**

4. Lean 4 is a high performance functional programming language in a category known as proof assistants, which rigorously check and prove mathematical theorems. With these generated Lean 4 snippets, we are able to automatically judge proof correctness instead of relying on LLM as a judge or other subjective techniques.

5.4.1 Tencent Decoupled reasoning

In July of 2025, scientists at *Tencent AI Lab* published a pivotal paper *Technical Report Towards Solving More Challenging IMO Problems via Decoupled Reasoning and Proving* [23], in which they created one of the first comprehensive systems dedicated to solving olympiad level math problems. This paper was published as a reaction to many of the fine tuned provers mentioned in subsection 5.2, and their performance on more complicated problems.

This "decoupled" approach, separating high-level reasoning from low-level formal proving, builds on foundational work like *Draft, Sketch, and Prove* (DSP) from 2023 [11]. The authors of

DSP were among the first to formalize a method for bridging the gap between human-readable proofs and machine-verifiable ones. Their key insight was to use an LLM to first generate an informal, natural-language proof (the "draft"). This draft was then used as a high-level guide to produce a formal proof "sketch," which outlines the key steps and intermediate lemmas in the formal language, but leaves the detailed proofs of each step incomplete (often filled with 'sorry' statements). This sketch then provides the crucial scaffolding for an automated prover to complete the proof. This methodology provided the conceptual groundwork for the more advanced *Reasoner* and *Prover* architectures that followed.

Dekoninck et al. [8] found the following:

Top-tier LLMs like Gemini 2.5 Pro can generate informal, natural-language solutions with over 80% accuracy after human verification. In contrast, the best formal prover Deepseek-Prover-V2 671B struggles to solve even 8% of the same problems directly.

Dekoninck et al. [8]

Why is this? Zhou et al. [23] gives a clear culprit as *Reinforcement Learning With Verifiable Rewards (RLVR)*:

This methodology [RLVR], used to train models like DeepSeek-Prover-v2 and Kimina, rewards only the final binary success or failure of the generated Lean code. This paradigm is fundamentally misaligned with the goal of bridging the reasoning-proving gap. Instead of rewarding hard-to-define, human-like strategies (the kind that achieve >80% informal success), the RLVR teaches a degenerated policy to maximize reward by any means necessary. It is incentivized to suppress its powerful, latent reasoning abilities in favor of heuristically decomposing goals into trivial sub-problems that can be solved by brute-forcing automatic tactics like ring, omega, or try. [23]

Essentially, these small provers fail on larger problems, because they try to brute-force their way towards a solution, which breaks down when the proofs get too long or complicated. We can see an example of this flawed reasoning in the appendix of the paper which gives a clear example of this

behavior on IMO Problem 1 using *DeepSeek-Prover-V2-671B* Ren et al. [15]. This problem asks us to find all functions $f : \mathbb{Z} \rightarrow \mathbb{Z}$ satisfying $f(2a) + 2f(b) = f(f(a + b))$ where a, b are integers.

The paper shows several code excerpts from attempted solutions from the model. In the first attempt, the model tries to brute force an enumeration of equations.

The model instantiates the functional equation on dozens of inputs, creating a large flat pool of algebraic identities, and then invokes tactics such as `ring_nf` and `linarith` in hopes of simplification. There is no effort to identify structure or extract reusable intermediate results. The tactic application is purely local and mechanical.

```

have h2 := hf 0 0
have h3 := hf 0 x
have h4 := hf x 0
have h5 := hf x (-x)
...
have h26 := hf (x + x) (-x)
ring_nf at h2 h3 h4 ... h26 ⊢

```

In the second attempt, the prover tries to assert the final form of the solution—namely $f(x) = 2x + f(0)$ —without having established why f must be linear or what motivates such a guess. It implicitly assumes the desired conclusion and attempts to work backward through aggressive simplification. This reveals a logical gap: the model never proves the Cauchy-like identity nor justifies why a linear form should even be expected.

```

have h29 : f x = 2 * x + f 0 := by
have h291 := hf x 0
...
ring_nf at h291 h292 ... ⊢
<;> linarith

```

The third attempt generates an even larger collection of equation instances, trying all possible combinations of inputs into the original functional equation, and then offloads the burden of reasoning onto a generic decision procedure like omega. Again, no insight is gained; the solution depends entirely on the capacity of low-level tactics to blindly traverse the search space. [23]

Essentially, the search space becomes too large for these models to ever come to a solution for these very complicated problems, and the models get stuck on things requiring other lemmas. The authors therefore crucially reasoned that a much better approach would be to separate the high level reasoning from proving smaller lemmas in the proof. We have already observed that general purpose off-the-shelf LLMs are much better at orchestrating, but it fails by skipping over steps and not being rigorous. On the other hand, we have proving models that are rigorous and minute in detail, but fail at large problems. The goal of this paper is to therefore connect these two approaches, having a high level reasoning model coupled with a lemma solving short formal proof model. This high level model is known as the *Reasoner* and the proving model is known as the *Prover*.

The reasoner has a very simple system prompt, telling it that it is to think step by step, and *decompose the original theorem into a sequence of smaller, logically coherent sub-theorems, each of which can be proved more easily* Zhou et al. [23] Also crucially included in this system prompt is a clear and expected output format:

Output Format for the Reasoner

- A brief explanation of your proof strategy (in natural language or Lean comments).
- A list of Lean 4 theorem declarations, each representing a sub-theorem, all starting with 'theorem XXX' and ending with ':= by sorry'.
- Ensure all sub-theorems are expressed using the same formal syntax and conventions as the input theorem.

[23] Thus the output of this *Reasoner* will be a list of unproven statements, which can be analyzed

and fed into the *Prover*. Using this simple approach, we can yield an entire end-to-end solution, that rectifies the negative aspect of both sets of models. This is the core first step towards building better systems for Automated Theorem Proving, and it yielded some spectacular results. Although they did not run their system on an established benchmark like *PutnamBench* or *MiniF2F*, they instead tested on a set of hard IMO problems from the exam offered in 2024. They were able to solve five of the previously unsolved problems. This marks a clear milestone of progress.

5.4.2 Seed-Prover

On August 1st, ByteDance AI labs released the paper, *Seed-Prover: Deep and Broad Reasoning for Automated Theorem Proving* Chen et al. [5]. It is important to note that the source code for this paper, as well as any system prompts, or low-level implementation details has not been published online, or verified by a third party. This paper achieved at the time SOTA performance on the *PutnamBench* benchmark, with performance of about 50% Chen et al. [5]. This system was the most complicated ATP system ever at the time of its release, and its release shepherded in a few very important advancements for the field as a whole.

1. **The ensembling of geometric and text-based subsystems:** *Seed-Prover* consists of two critical subcomponents, *Seed-Prover* and *Seed-Geometry*. It is not specified what the mechanism is for the assignment of problems from one subsystem over another, but this specialization of proving type provides clear benefits in performance over previous techniques. For a deeper explanation of Seed-Geometry, refer to [subsubsection 5.3.2](#). Chen et al. [5]
2. **Conjecture Proposing as opposed to Draft, Sketch, Prove** Instead of following a pre-defined informal proof, as seen in "Draft, Sketch, Prove" (DSP) methods Jiang et al. [11], *Seed-Prover* introduces a dynamic "conjecture proposing" mechanism. This system actively generates and evaluates intermediate lemmas and conjectures during the proof search. This allows the prover to explore more diverse proof paths and discover novel intermediate steps that may not be present in a human-written draft, enabling it to solve problems without existing informal solutions. Chen et al. [5]

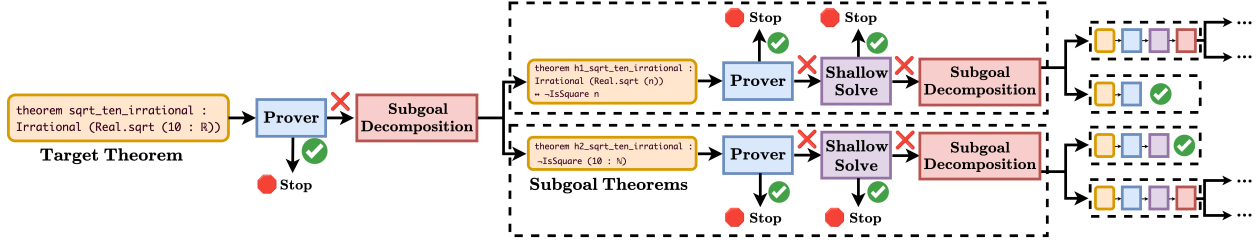


Figure 1: Hilbert architecture and overview. Varambally et al. [19]

3. **Test time scaling** *Seed-Prover* leverages significant computational resources at inference time to enhance performance. This includes techniques such as generating a large number of potential proof attempts (self-consistency) and employing sophisticated search algorithms (like MCTS) guided by the model’s heuristics. By scaling the search budget and sampling diversity at test time, the system can systematically explore a much larger proof space than a single, greedy decoding pass, significantly increasing its problem-solving success rate. Chen et al. [5]

5.4.3 Hilbert

In the prior approach by Zhou et al. [23], we saw a single model create a high level overview for the entire problem at once, in a rather linear fashion. The model simply generates a template of what lemmas need to be proven, which are then handed off to the *Prover* for formal proof generation. This approach, however, only works assuming that the problem is simple enough that it can be decomposed into lemmas in a single pass, and that the problem can be solved by the *Prover* in a single pass. Except, there are cases where this may not be true, for particularly hard problems like those on *PutnamBench*, it may require so much reasoning that solving these problems is quite challenging for a single decomposition step.

So instead of having a fixed pathway to decompose our problem, what if we took a more recursive approach? This is exactly what Hilbert [19], an ATP system created by Apple Labs, does. Released in late September of this year, Hilbert outperforms all other benchmarks, achieving SOTA performance on everything thrown at it. Particularly of interest, it was one of only two systems that

were able to solve over 25% of the problems on *PutnamBench*. In fact, it was able to achieve 70% accuracy in generating a verified formal proof in Lean 4. Given exam grades from the past few years, assuming that a verified Lean 4 submission would be considered a complete proof with a 10/10 score, and that there was no partial credit, you would expect on average the model to score around an 84/120 on the exam, which would easily place it in the top 20 participants, and assuming that there was any partial credit given, it would not be outside the realm of possibility for this system to achieve gold-level results on one of these exams.

The algorithm for generating these proofs is rather complex in comparison to that of the Tencent paper. We begin by inputting the skeleton statement that we want to prove as a Lean 4 skeleton, with a `sorry` statement at the end. The model then attempts a simple shallow solve of the proof, with the *Reasoner*. After 5 failed attempts, it executes the following procedure:

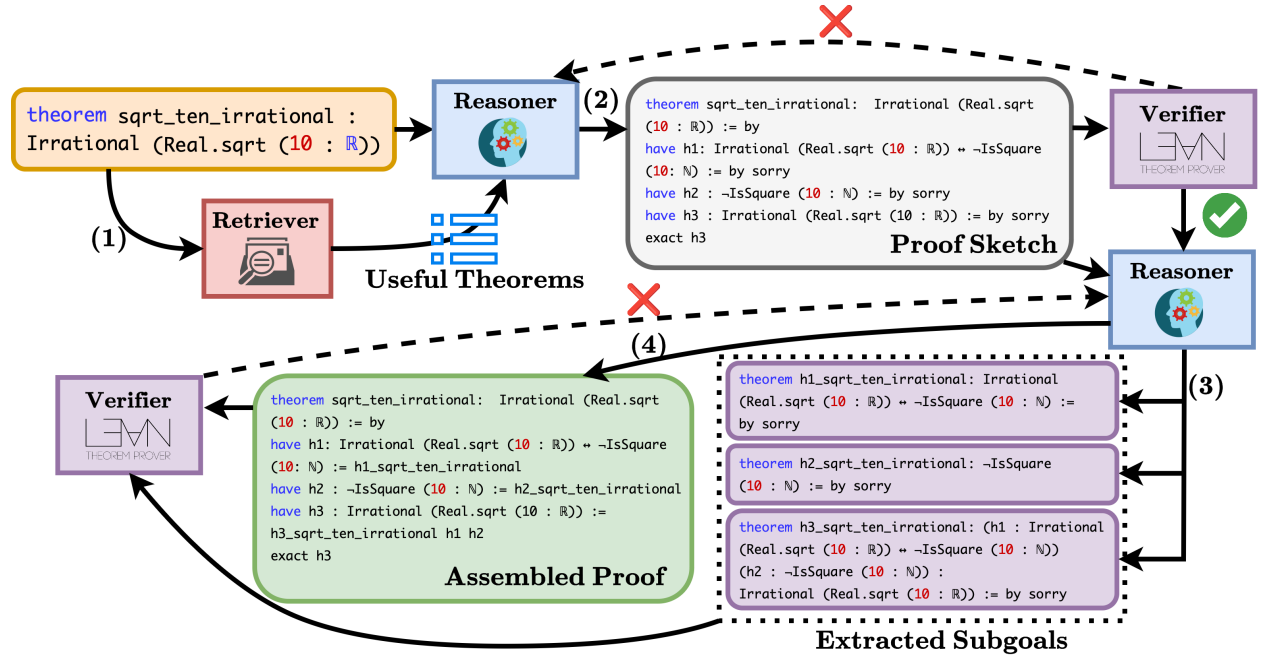


Figure 2: In depth guide to subgoal decomposition. Varambally et al. [19]

Initial Subgoal Decomposition

1. Instead of starting with a single decomposition pass, the *Retriever* takes the given theorem

and performs a semantic search on a Lean 4 library called **Mathlib**. The *Reasoner* is then queried again to select only the top relevant theorems for use in the proof. [19]

2. The reasoner is prompted to produce a detailed informal proof using the retrieved theorems. [19]
3. With this proof supplied in-context, the Reasoner is asked to generate a Lean 4 proof sketch that decomposes the problem into simpler subproblems represented as **have** statements. [19]
4. From this point, all subgoals are initially filled with **sorry** statements. [19]
5. The shallow proof is verified using the *Verifier*, and any feedback is leveraged into the proof. This sketch attempt is run a maximum of 4 times, before being abandoned. [19]
6. The *Reasoner* then extracts these subgoals from the proof sketch, converting each **have** statement into an independent theorem statement. Relevant context from the original problem and preceding subgoals is added to these new subgoals to make them self-contained. [19]
7. Each new subgoal theorem is checked by the *Verifier* for syntactical correctness. The *Reasoner* iteratively corrects any errors by using the verifier’s feedback. [19]
8. Finally, the *Reasoner* produces the assembled proof. It takes the original proof sketch and replaces each **sorry** placeholder with a direct call to the corresponding subgoal theorem that was just extracted. This complete, assembled proof structure is then verified one last time to ensure the overall logic is sound. [19]

Subgoal verification

We now have an assembled and valid proof sketch. The system now begins verifying and proving each extracted subgoal. We execute the following procedure on each subgoal:

1. The first attempt is made by the *Prover*, a specialized LLM, which generates four candidate proofs. If any proof is validated, it is accepted, and the system moves to the next subgoal.[19]

1. If the *Prover* fails, the *Reasoner* is prompted to evaluate the subgoal’s mathematical correctness and provability. This step acts as a filter to prevent the system from wasting computational resources on incorrectly formulated or unprovable subgoals. [19]
2. If the *Reasoner* identifies an issue (e.g., a mathematical error, missing hypotheses, or problems with the Lean type system), the subgoal is flagged. The system then discards the current proof sketch and returns to the initial sketch generation step (Section 3.2.1) using the feedback to create a corrected sketch. [19]
3. If the subgoal is deemed correct but was not solved by the *Prover*, the system attempts a ”shallow solve.” The *Reasoner* is employed to write a short formal proof, retrieving theorems from Mathlib as needed. It iteratively refines its proof using *Verifier* feedback for up to six passes. [19]
4. This ”shallow solve” is computationally bounded: the process is terminated if a proof attempt exceeds 30 lines (indicating need for further decomposition) or if it fails to produce a valid proof after six attempts. [19]
5. If a subgoal remains unproven after all previous steps, the system applies recursive decomposition. The problematic subgoal is treated as a new top-level problem and is fed back into the full subgoal decomposition process (Section 3.2.1) to be broken down further. [19]

This recursive process continues until either the subgoal is proven or a maximum recursion depth is reached. Once all subgoals are successfully proven, their proofs are concatenated with the assembled proof outline to generate the final, complete proof for the target theorem.

It is clear from the published results that Hilbert is a massive leap forward over [23]. Achieving an astounding 99.2% accuracy on the *Mini-F2F* benchmark and 70% on *PutnamBench*, Hilbert is clearly far ahead of the other open-source competition.

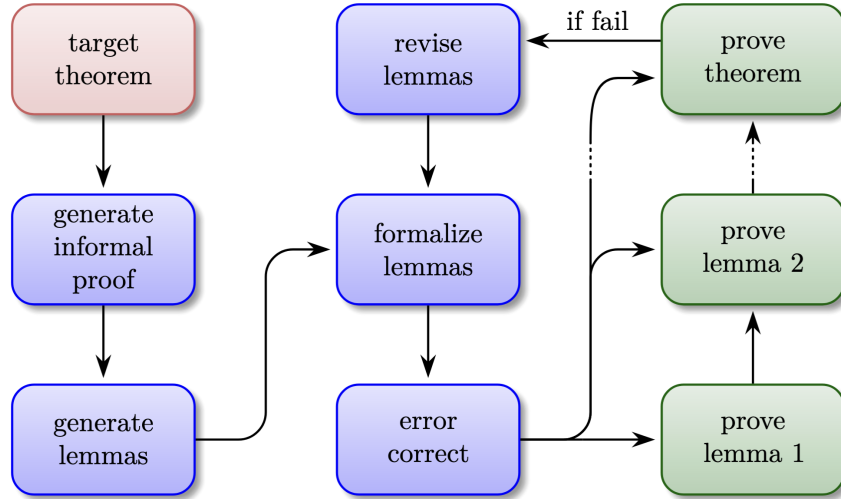


Figure 3: The target theorem (red) is processed through a pipeline of natural language queries (blue), resulting in a series of lemmas which are processed by the search algorithm (green). If the theorem remains unproven at the end, the unproven lemmas are revised and the procedure is repeated. Achim et al. [1]

5.4.4 Aristotle

In July 2025, the AI lab Harmonic introduced **Aristotle**, an AI system that achieved a gold-medal-equivalent performance on the 2025 International Mathematical Olympiad (IMO) by providing formally verified Lean 4 solutions to five out of the six problems. This performance was notable as the only human step was the initial transcription of the informal problem statements into formal Lean statements; all intermediate lemmas were autoformalized by the system.

Aristotle’s architecture is fundamentally different from the LLM-driven decomposition pipelines of the Tencent paper and Hilbert. Instead of relying on an LLM to recursively break down a problem until a simple ”Prover” model can solve it, Aristotle is built around a powerful, specialized search algorithm as its core engine, the system integrates three main components [1]:

1. **A Lean Proof Search Algorithm:** This is the system’s ”most fundamental component”. It is not a simple LLM call but a **highly parallel Monte Carlo Graph Search (MCGS)**. This algorithm’s job is to take a Lean proof sketch (with sorry statements) and find a complete proof by predicting Lean tactics conditional on the proof state and history.

2. **A Lemma-based Informal Reasoning System:** This component functions as a high-level planner that boosts the performance of the MCGS. As shown in of the paper, this system first generates an informal, natural language proof of the target theorem. It then breaks this proof down into a sequence of lemmas, formalizes these lemmas into Lean, and uses feedback from the Lean REPL to correct errors. These proven lemmas are then fed as context to the MCGS, effectively guiding its search.[1]
3. **A Dedicated Geometry Solver:** Recognizing the unique challenges of geometry, Aristotle uses a separate solver called **Yuclid**. This is a C++ Deductive Database and Algebraic Reasoning (DDAR) engine, which the authors claim is up to 500x faster than the one used in AlphaGeometry-1 [1]

While Hilbert uses recursion to break down a failed subgoal, Aristotle uses an iterative feedback loop between its informal and formal systems. If the MCGS fails to prove the main theorem using the first pass of generated lemmas, the system identifies which lemmas were proven and which remain unproven [1]. This formal feedback is passed back to the informal reasoning system, which is then prompted to revise the proof strategy, either by fixing flawed lemmas or breaking them into more granular steps, in a manner quite similar to Hilbert [1].

Furthermore, Aristotle employs a novel technique called *Test-Time Training* (TTT) [1]. If the system fails to solve a problem after its initial attempts, it does not just try a new prompt; it retrains its model on the search traces extracted from its own failed attempts [1]. This allows the model to learn from its specific failures at inference time, specializing itself to the difficult problem at hand and "accelerating the rate at which Aristotle processed lemmas" during the IMO evaluation [1]. This dynamic, self-improving search architecture stands in sharp contrast to the more static, prompt-driven decomposition of other systems.

5.4.5 Ax-Prover

Released in November 2025, **Ax-Prover** presents a different architectural philosophy, positioning itself as a "lightweight agentic workflow" designed to counteract the limitations of heavyweight,

specialized provers [3]. The authors argue that while specialized models like DeepSeek-Prover achieve high performance on specific benchmarks, they suffer from three key limitations: (1) they fail to generalize to new scientific domains beyond their training data [3]; (2) they are "brittle" to changes in the fast-evolving Mathlib library, requiring constant, expensive retraining [3]; and (3) they are "burdensome" to deploy, often requiring high-performance-computing clusters (e.g., 8x H200 GPUs for DS-Prover) that are inaccessible to most researchers [3].

Ax-Prover's solution is to "sidestep" specialized models entirely. It does not use any domain-specific training. Instead, it uses a multi-agent system of general-purpose LLMs (e.g., Claude Sonnet 4.5) equipped with a suite of "Lean tools" via the Model Context Protocol (MCP). The architecture consists of three agents:

1. **The Orchestrator:** A scheduler that assigns tasks and routes feedback.[3]
2. **The Prover:** The "constructive core" of the system. It emulates a human workflow: it first analyzes an unproven theorem (marked with `sorry`) and generates a high-level, natural language "proof sketch". It then formalizes this sketch into a sequence of `have ... by sorry` statements, effectively breaking the problem into manageable steps. Crucially, it then solves each step sequentially, using its tools in a "tight feedback loop". It uses `lean_leansearch` to find relevant lemmas in Mathlib, `lean_goal` to inspect the proof state, and `lean_diagnostic_messages` to verify its own work before moving on [3].
3. **The Verifier:** A final "gatekeeper" that performs a last check to ensure the final proof is complete and contains no errors or `sorry` placeholders [3].

To prove their "generalizability" argument, the authors introduced two new benchmarks: **AbstractAlgebra** (research-level problems) and **Quantum Theorems** (physics problems). On these, Ax-Prover significantly outperformed the specialized provers. On AbstractAlgebra, it scored 64% to DS-Prover's 24%, and on Quantum Theorems, it achieved 96% to DS-Prover's 61%. The paper notes this is because the Quantum dataset used "custom definitions" not found in Mathlib; the

specialized provers, over-specialized on Mathlib, failed, whereas Ax-Prover’s general-purpose LLM could ”flexibly incorporate” the new concepts [3].

Dataset	Subset	Ax-Prover	Sonnet	DS-Prover	Kimina
NuminaMath-LEAN	solved-K	81%	7%	48%	100% [†]
	solved-H	47%	8%	14%	0% [†]
	unsolved	26%	1%	18%	0% [†]
	total	51%	5%	28%	31%
AbstractAlgebra	easy	72%	10%	26%	12%
	intermediate	56%	6%	22%	14%
	total	64%	8%	24%	13%
QuantumTheorems	easy	100%	54%	88%	72%
	intermediate	92%	18%	48%	34%
	total	96%	40%	61%	57%

Table 2: Models’ performance on NuminaMath-LEAN, AA, and QT. [†] The results on NuminaMath-LEAN for Kimina are reported from [?], and were obtained during its RL training phase with, on average, pass@68. Breen et al. [3]

On PutnamBench, Ax-Prover’s result of 14% accuracy (92 problems) appears low compared to Hilbert’s 70%. However, the authors highlight this 14% was achieved with a ”much leaner compute budget” and using pass@1 (defined as a single, multi-step agentic run), whereas systems like Hilbert used an ‘avg. pass@1840’ and DeepSeek-Prover used ‘pass@1024’. Ax-Prover thus presents itself as a more accessible and generalizable alternative, designed to be a ”researcher-friendly assistant” rather than a heavyweight autonomous system [3].

5.4.6 AlphaProof

On November 12, 2025, Google AI Labs published *Olympiad-level formal mathematical reasoning with reinforcement learning*, unveiling *AlphaProof*. This system established a new state-of-the-art (SOTA) on multiple formal reasoning benchmarks, notably increasing accuracy on the Mini-F2F dataset from 99.4% to 99.6%. The architecture represents the culmination of several incremental advancements in neuro-symbolic computing described in prior literature [10].

In contrast to systems reliant solely on Large Language Model (LLM) decomposition, AlphaProof operates as an **AlphaZero-inspired agent**. It is designed to construct formal proofs within the Lean

theorem prover via large-scale reinforcement learning (RL). The architectural framework integrates a pre-trained language model—specifically, a 3-billion parameter encoder-decoder transformer designated as the “proof network”—with a Monte Carlo Tree Search (MCTS) algorithm. This hybrid approach addresses the infinite action space of formal mathematics by using the policy network to generate candidate tactics and a value network to estimate proof viability, while the Lean environment serves as a deterministic verifier to ensure logical soundness.

A critical bottleneck in training such systems is the scarcity of formal mathematical data. AlphaProof circumvents this limitation through a novel **auto-formalization pipeline**. The methodology employs a specialized Gemini-based LLM to translate approximately one million informal, natural-language problems into formal Lean statements. Through an iterative validation process, this yields a massive synthetic dataset of approximately 80 million formal problems. This corpus functions as the primary curriculum for the RL loop, allowing the agent to learn via “self-generated experience.” By attempting to prove or disprove millions of synthetic statements and utilizing the verified outcomes as feedback, the system iteratively refines its policy and value networks.

The efficacy of this system was empirically demonstrated at the 2024 International Mathematical Olympiad (IMO). Deployed as a neuro-symbolic solver in conjunction with AlphaGeometry 2, the system achieved a performance benchmark equivalent to a silver medal. AlphaProof successfully solved three of the five non-geometry problems (P1, P2, and P6). Notably, the system solved Problem 6, traditionally the competition’s most challenging problem, which was solved by only five human contestants. The combined score of 28 points represents the first instance of an AI system achieving medal-level performance at the IMO [10].

6 Problems with Benchmarks in ATP

Benchmarks are a commonly used way to gauge and compare the performance of different LLM systems. They are essentially tests of problems on a given topic, with various methods of determining whether an answer is correct or not. However, there exists a severe problem in the context of ATP,

Data Contamination. Data Contamination arises when the problems themselves end up in the training sets of the models that are used in these systems. For example, let’s say that in the corpus of Gemini 2.5 Pro were all of the pages of MathStackExchange indexed by Google. If any of these were to contain solutions to any problems on any of the Putnam exams, their solutions would end up in the weights and parameters of the model during its pre-training phase.

When this ”contaminated” model is subsequently evaluated on that same Putnam exam, its performance is no longer a genuine measure of its reasoning or problem-solving abilities. Instead of deducing a solution, the model is, in effect, retrieving a memorized answer or a close statistical approximation of one it has already seen.

This artificially inflates the model’s benchmark scores, rendering the evaluation invalid. The test, which is designed to gauge generalization and novel deductive skill, inadvertently becomes a test of memorization. This is a critical flaw, as it provides a false signal of progress in the field and misleads researchers about the true capabilities of the LLM, especially in specialized domains like ATP where genuine, novel reasoning is the ultimate goal.

6.1 Benchmarks Commonly Used for ATP

The evaluation of automated mathematical reasoning systems, particularly those targeting formal proof generation, has converged on a set of standardized benchmarks. Among these, two benchmarks are the most important for claims of SOTA performance: *MiniF2F* and *PutnamBench*.

1. **MiniF2F** Introduced by Zheng et al. [22], MiniF2F (standing for ”formal-to-formal”) is a widely adopted benchmark comprising 488 problem statements. These problems are sourced from various high-school level mathematics competitions, most notably the American Invitational Mathematics Examination (AIME), the International Mathematical Olympiad (IMO), and the American Mathematics Competitions (AMC). It has served as a primary standard for evaluating systems on creative, pre-university-level reasoning.
2. **PutnamBench:** PutnamBench was compiled due to the increasingly strong performance of

systems on MiniF2F and related benchmarks. PutnamBench was introduced by Tsoukalas et al. [18]. It consists of 640 problems drawn from the William Lowell Putnam Mathematical Competition (1962-2024), the premier undergraduate-level mathematics competition in North America. Critically, PutnamBench provides complete source code/formalization for every problem in Lean 4. This inclusion of formal statements has made it the primary and most challenging benchmark for the formal ATP systems discussed in this thesis, such as Hilbert. Consequently, achieving high performance, or "saturating" this benchmark, has become a principal goal for top research labs.

6.2 Data Contamination for all systems

The issue of data contamination extends far beyond the specialized domain of Automated Theorem Proving (ATP). It is a pervasive problem across the entire field of large language model (LLM) evaluation. When benchmark datasets, which are intended to be unseen test sets, are included in the massive pre-training corpus, the resulting performance metrics become unreliable. This overlap undermines the fundamental purpose of benchmarking, as it becomes impossible to distinguish between a model's genuine ability to generalize and its capacity for simple memorization [7].

This problem has significantly tarnished many widely-used benchmarks, leading to exaggerated claims of model capability and misleading signals of progress in AI reasoning. Consequently, a critical area of research has emerged, focused not just on creating new, "clean" benchmarks, but on developing robust methods to detect contamination in existing ones.

6.3 Data Contamination in ATP

There are strong and justified concerns about the pervasiveness of Data Contamination in the field of ATP. In 2025, a website launched called matharena.ai, which tracks the performance of Off-The-Shelf LLMs on new problems published from math competitions that came out after their training cutoffs. This website was published in conjunction with other research done in Balunović et al. [2]. The authors found a clear observed drop-off in performance on problems from exams

authored after the training cutoff of these models, illustrating clear evidence of Data Contamination.

In a very interesting study, researchers found a workaround to the problems faced by public benchmarks, they constructed an entirely new benchmark on old Soviet and eastern European national math olympiads, which had several important key characteristics. Khatibi et al. [12]

1. The contents of these exams had never been put on the internet, and therefore it would be impossible for them to be in the training corpus for these Off-The-Shelf models.
2. The benchmark is curated from under-circulated regional and national Olympiads of Eastern Europe and the countries from the former Soviet Union.
3. The problems feature a comparable difficulty to the International Mathematics Olympiad (IMO).
4. They are renowned for demanding nonstandard problem-solving techniques.

As a result of all the features above, good performance on this benchmark would be a very good signal for these models, and for these systems as a whole. Unfortunately, this is not what the authors observed. The authors observed that even state-of-the-art LLMs exhibit a notable performance decline on the EEFSUVA benchmark relative to other Olympiad-style benchmarks. [12] You can see this in Table 1.

6.4 Reproducibility Concerns more broadly

Model	HHMT	SMT	BRUMO	IMO	USMO	EEFSUVA
Gemini 2.5 Pro	82.50%	84.91%	90.00%	31.55%	24.40%	0%
GPT-5 Thinking/High	88.33%	91.98%	91.67%	38.10%	n/a	35.89%

Table 3: EEFSUVA compared to other benchmarks. Khatibi et al. [12]

You may notice that there were no results in Table 6.4 for the proving systems mentioned in subsection 5.4 for the MathArena paper specifically. This is due to a shocking lack of replication in the field of ATP. To reproduce one of these systems from subsection 5.4 would be an expensive

undertaking, both in terms of time required to recreate such a system, and also in the price of API calls. Thus, there is little incentive for third-party labs to recreate the systems used for a fair evaluation. This is not to mention that for many of the systems, for example, Aristotle, the source code and exact methodology is proprietary, and therefore it would be impossible for a third party to reproduce these results. As a result, there are many unanswered questions about the exact validity of these purported results. It seems highly unlikely that these claims are entirely falsified, but they have not been verified by any third parties whatsoever.

7 Future directions

The preceding sections highlighted numerous unanswered questions regarding the performance of ATP systems. This section proposes future research directions and identifies key open questions in the field.

7.1 Documenting Scalability in System-Agnostic Techniques

Addressing the open questions surrounding ATP performance requires new approaches. This subsection focuses on one promising research direction: system-agnostic techniques that can be applied broadly to enhance performance irrespective of the specific model or system being used. It might be interesting to re-examine Hilbert [19] or the Tencent Paper [23] by adjusting many of the hyperparameters that apply to these systems, and documenting how changing them affects performance. Here a few possible directions for this research:

1. Determining the importance of the N parameter for Pass@ N , is there ever a level off in performance when increasing the number of trials? It would be interesting to see how performance levels off, and try to model this as an equation with some sort of scaling law. However, as the number of trials grows arbitrarily large, you would expect accuracy to reach 100%, as a corollary of the *Infinite Monkey Theorem*, where in this case, the Monkey is the LLM. (see Brown et al. [4])

2. Determining which model is the best *Prover* and which is the best *Reasoner*, and which is the best *Verifier*. This would be comparatively easy to test, since there are only about 4-5 models that would be considered competitive for each category.
3. Making a specialized dataset of human-generated solutions for difficult examples that models fail to solve, enabling a more targeted analysis of model weaknesses.

7.2 Reproducing Papers and Comparing Results

Even though Aristotle is closed source, it is accessible through a paid API platform. It would be very interesting therefore to test it and the other models on the input Lean 4 formalizations for all of the questions on PutnamBench, and then see what sort of results each system was able to achieve. This would be rather expensive, since Aristotle is a closed source proprietary model, but it would definitely be feasible to test a smaller benchmark to compare performance. This could then be made into a website as well, that shows the results of these models and determines which is the best using an elo ranking system, similar to *MathArena*.

7.3 Implementing papers that propose better benchmarking

The BEYONDBENCH framework Srivastava et al. [16] offers a solution to data contamination by proposing an evaluation method based on algorithmic problem generation. Instead of using a fixed set of questions, this approach dynamically creates new, mathematically-grounded problems on the fly, ensuring that test instances remain fresh and uncontaminated.

The framework spans a wide range of tasks, from basic arithmetic to NP-complete problems, with each task category capable of generating a vast number of unique instances (e.g., $> 10^{15}$). Solutions are verified deterministically (using Lean), providing a robust measure of a model's genuine reasoning capabilities rather than its recall. A future research direction would be to implement a similar algorithmic generation framework, specifically for ATP. This would allow for a rigorous, contamination-resistant evaluation of systems, testing their performance as problem

complexity scales and providing a clearer signal of their true problem-solving abilities. Also, it would allow for data-augmentation for training.

Bibliography

- [1] Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, Mathis F  d  rico, Sergei Gukov, Daniel Halpern-Leistner, Kirsten Henningsgard, Yury Kudryashov, Alexander Meiburg, Martin Michelsen, Riley Patterson, Eric Rodriguez, Laura Scharff, Vikram Shanker, Vladimir Sicca, Hari Sowrirajan, Aidan Swope, Matyas Tamas, Vlad Tenev, Jonathan Thomm, Harold Williams, and Lawrence Wu. Aristotle: Imo-level automated theorem proving, 2025. URL <https://arxiv.org/abs/2510.01346>.
- [2] Mislav Balunovi  , Jasper Dekoninck, Ivo Petrov, Nikola Jovanovi  , and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark*, 2025.
- [3] Benjamin Breen, Marco Del Tredici, Jacob McCarran, Javier Aspuru Mijares, Weichen Winston Yin, Kfir Sulimany, Jacob M. Taylor, Frank H. L. Koppens, and Dirk Englund. Ax-prover: A deep reasoning agentic framework for theorem proving in mathematics and quantum physics, 2025. URL <https://arxiv.org/abs/2510.12787>.
- [4] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher R  , and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- [5] Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving, 2025. URL <https://arxiv.org/abs/2507.23726>.
- [6] Yuri Chervonyi, Trieu H. Trinh, Miroslav Ol    k, Xiaomeng Yang, Hoang Nguyen, Marcelo

- Menegali, Junehyuk Jung, Vikas Verma, Quoc V. Le, and Thang Luong. Gold-medalist performance in solving olympiad geometry with alphageometry2, 2025. URL <https://arxiv.org/abs/2502.03544>.
- [7] Hyeon Kyu Choi, Maxim Khanov, Hongxin Wei, and Yixuan Li. How contaminated is your benchmark? measuring dataset leakage in large language models with kernel divergence. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=wVDR2qmE28>.
- [8] Jasper Dekoninck, Ivo Petrov, Kristian Minchev, Mislav Balunovic, Martin Vechev, Miroslav Marinov, Maria Drencheva, Lyuba Konova, Milen Shumanov, Kaloyan Tsvetkov, Nikolay Drenchev, Lazar Todorov, Kalina Nikolova, Nikolay Georgiev, Vanesa Kalinkova, and Margulan Ismoldayev. The open proof corpus: A large-scale study of llm-generated mathematical proofs, 2025. URL <https://arxiv.org/abs/2506.21621>.
- [9] Jiaxing Guo, Wenjie Yang, Shengzhong Zhang, Tongshan Xu, Lun Du, Da Zheng, and Zengfeng Huang. Right is not enough: The pitfalls of outcome supervision in training llms for math reasoning, 2025. URL <https://arxiv.org/abs/2506.06877>.
- [10] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, Ottavia Bertolli, Tom Zahavy, Amol Mandhane, Jessica Yung, Iuliya Beloshapka, Borja Ibarz, Vivek Veeriah, Lei Yu, Oliver Nash, Paul Lezeau, Salvatore Mercuri, Calle Sönne, Bhavik Mehta, Alex Davies, Daniel Zheng, Fabian Pedregosa, Yin Li, Ingrid von Glehn, Mark Rowland, Samuel Albanie, Ameya Velingker, Simon Schmitt, Edward Lockhart, Edward Hughes, Henryk Michalewski, Nicolas Sonnerat, Demis Hassabis, Pushmeet Kohli, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y. URL <https://doi.org/10.1038/s41586-025-09833-y>.
- [11] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik,

- Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs, 2023. URL <https://arxiv.org/abs/2210.12283>.
- [12] Nicole N Khatibi, Daniil A. Radamovich, and Michael P. Brenner. Eefsuva: A new mathematical olympiad benchmark, 2025. URL <https://arxiv.org/abs/2510.01227>.
- [13] Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, Jiayun Wu, Jiri Gesi, Ximing Lu, David Acuna, Kaiyu Yang, Hongzhou Lin, Yejin Choi, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction, 2025. URL <https://arxiv.org/abs/2508.03613>.
- [14] Ivo Petrov, Jasper Dekoninck, Lyuben Baltadzhiev, Maria Drencheva, Kristian Minchev, Mislav Balunović, Nikola Jovanović, and Martin Vechev. Proof or bluff? evaluating llms on 2025 usa math olympiad, 2025. URL <https://arxiv.org/abs/2503.21934>.
- [15] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025. URL <https://arxiv.org/abs/2504.21801>.
- [16] Gaurav Srivastava, Aafiya Hussain, Zhenyu Bi, Swastik Roy, Priya Pitre, Meng Lu, Morteza Ziyadi, and Xuan Wang. Beyondbench: Benchmark-free evaluation of reasoning in language models, 2025. URL <https://arxiv.org/abs/2509.24210>.
- [17] Yuchen Tian, Ruiyuan Huang, Xuanwu Wang, Jing Ma, Zengfeng Huang, Ziyang Luo, Hongzhan Lin, Da Zheng, and Lun Du. Evolver: Advancing automated theorem proving by evolving formalized problems via symmetry and difficulty, 2025. URL <https://arxiv.org/abs/2510.00732>.

- [18] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL <https://arxiv.org/abs/2407.11214>.
- [19] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning, 2025. URL <https://arxiv.org/abs/2509.22819>.
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- [21] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models, 2023. URL <https://arxiv.org/abs/2306.15626>.
- [22] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics, 2022. URL <https://arxiv.org/abs/2109.00110>.
- [23] Yichi Zhou, Jianqiu Zhao, Yongxin Zhang, Bohan Wang, Siran Wang, Luoxin Chen, Jiahui Wang, Haowei Chen, Allan Jie, Xinbo Zhang, Haocheng Wang, Luong Trung, Rong Ye, Phan Nhat Hoang, Huishuai Zhang, Peng Sun, and Hang Li. Solving formal math problems by decomposition and iterative reflection, 2025. URL <https://arxiv.org/abs/2507.15225>.